

Tema 5 - Estructura de un proyecto Python

Francisco Gortázar

¿Qué es un módulo?

- Un módulo es un archivo que contiene código (funciones, clases o variables)
- Permite organizar el software en bloques lógicos independientes
- Cada módulo define su propio espacio de nombres (namespace)
 - Nombres definidos en él no entran en conflicto con otros nombres en otras partes del programa

Importación de módulos

- Para utilizar el contenido de un módulo es necesario importarlo desde donde se vaya a utilizar
- Para importar un módulo se utiliza la sentencia `import`

Importación de módulos

- Existen diversas variantes:
 - **Importación completa:** Se importa todo el módulo y se accede a sus funciones mediante la notación de punto

```
import pizza  
  
pizza.make_pizza()
```

Importación de módulos

- Existen diversas variantes:
 - **Importación específica:** Permite importar solo los elementos necesarios directamente al espacio de nombres actual.

```
from pizza import make_pizza  
  
make_pizza()
```

Importación de módulos

- Existen diversas variantes:
 - **Uso de alias:** Se puede asignar un nombre alternativo al módulo o a la función para mayor brevedad o evitar conflictos.

```
import pizza as p  
  
p.make_pizza()
```

```
from pizza import make_pizza as mp  
  
mp()
```

Definición de módulos

- Un módulo es cualquier archivo con extensión .py
- Para que el sistema lo reconozca, el módulo debe estar guardado en el mismo directorio que el programa principal o en una ruta contenida en sys.path
- Ejemplo de archivo **pizza.py**:

```
project_root/  
  src  
    app.py  
    pizza.py
```

```
def make_pizza(size, toppings):
    """Resume la pizza que estamos preparando."""
    print(f"Preparando pizza {size}, ingredientes:")
    for topping in toppings:
        print(f"- {topping}")
```

Ejecución de módulos como programa

- Cuando se importa o ejecuta un módulo, se ejecutan todas las instrucciones que contenga
- Dado el siguiente módulo **fibonacci.py**:

```
def fibonacci(n):
    if n == 0:
        return 0
    elif n == 1:
        return 1
    else:
        return fibonacci(n-1) + fibonacci(n-2)

print("Hey... ")
```

Ejecución de módulos como programa

- Cuando lo importamos desde otro módulo con `import fibonacci`:

```
import fibonacci
print("Wow")
```

- Salida:

```
Hey...
Wow
```

Ejecución de módulos como programa

- Para gestionar esta situación Python utiliza una variable interna llamada `__name__` que determina el contexto de ejecución
 - Cuando un archivo se ejecuta directamente, `__name__` se establece en `"__main__"`.
 - Si el archivo es importado como módulo, `__name__` toma el nombre del archivo (`"fibonacci"` en este caso)
- Se utiliza un bloque `if __name__ == "__main__":` para:
 - Diferenciar dentro del módulo cuándo se importa como módulo y cuándo se ejecuta como un programa
 - Realizar inicializaciones cuando se usa como un programa

Ejecución de módulos como programa

- Ejemplo (permite ejecutar el módulo como un programa):

```
def fibonacci(n):
    if n == 0:
        return 0
    elif n == 1:
        return 1
    else:
        return fibonacci(n-1) + fibonacci(n-2)

if __name__ == "__main__":
    import sys
    if len(sys.argv) != 2:
        print("Usage: python fibonacci.py <n>")
        sys.exit(1)
    print(fibonacci(int(sys.argv[1])))
```

Definición de paquetes

- Un paquete es una forma de estructurar módulos en una jerarquía organizada
- Un paquete es un directorio que contiene módulos y un archivo llamado `__init__.py` que indica a Python que ese directorio debe ser tratado como un paquete.

```

project_root/
  requirements.txt      <-- Dependencias del proyecto
  README.md            <-- Descripción del proyecto
  src                  <-- Carpeta principal del código fuente
    __init__.py        <-- Identifica al directorio como un paquete jerárquico
    app.py             <-- Módulo con la lógica principal o punto de entrada
    package_A          <-- Un paquete
      __init__.py      <-- Identifica al directorio como un paquete jerárquico
      modulo_x.py      <-- Módulo dentro de la subjerarquía

```

Definición de paquetes

- Los paquetes pueden anidarse:

```

project_root/
  requirements.txt      <-- Dependencias del proyecto
  README.md            <-- Descripción del proyecto
  src                  <-- Carpeta principal del código fuente
    __init__.py        <-- Identifica al directorio como un paquete jerárquico
    app.py             <-- Módulo con la lógica principal o punto de entrada
    package_A          <-- Un paquete
      __init__.py      <-- Identifica al directorio como un paquete jerárquico
      modulo_x.py      <-- Módulo dentro de la subjerarquía

    package_B          <-- package_B es un subpaquete de package_A
      __init__.py      <-- Identifica al directorio como un paquete jerárquico
      modulo_y.py      <-- Módulo dentro de la subjerarquía

```

Definición de paquetes

- Para acceder a un módulo dentro de un paquete, se utiliza la notación de punto para navegar por la estructura de directorios.
- Desde `app.py` el `modulo_x` del paquete `package_A` se importaría así:

```

1 import package_A.module_x
2
3 print (package_A.module_x.hola())
4
5 from package_A import module_x
6
7 print (module_x.hola())

```

```

8
9 from package_A.module_x import hola
10
11 print (hola())

```

```

project_root/
  src
    __init__.py
  |   app.py
  |   package_A
  |       __init__.py
  |       modulo_x.py
  |
  |   package_B
  |       __init__.py
  |       modulo_y.py

```

Definición de paquetes

- Para acceder a un módulo dentro de un **subpaquete**, se utiliza la notación de punto para navegar por la estructura de directorios.
- Desde app.py el modulo_y del subpaquete package_B se importaría así:

```

1 import package_A.package_B.module_y
2
3 print (package_A.package_B.module_y.adios())
4
5 from package_A.package_B import module_y
6
7 print (module_y.adios())
8
9 from package_A.package_B.module_y import adios
10
11 print(adios())

```

```

project_root/
  src
    __init__.py
  |   app.py
  |   package_A
  |       __init__.py

```

```
|         modulo_x.py
|
|         package_B
|             __init__.py
|             modulo_y.py
```

Publicación de paquetes

- Los paquetes pueden ser distribuidos para que otros desarrolladores los utilicen
- El lugar estándar para publicar paquetes Python de código abierto es el [Python Package Index \(PyPI\)](#)
- Para distribuir y publicar paquetes se suelen utilizar herramientas:
 - setuptools,
 - wheel,
 - twine,
 - o gestores modernos como Poetry
- Una vez publicado, el paquete se instala con el comando

```
pip install <nombre>
```

Estructura de un proyecto Python

- La estructura habitual para proyectos Python es la siguiente:

```
1  mi_proyecto/          <-- Directorio raíz del proyecto
2
3  .gitignore            <-- Indica qué archivos ignorar en el control de versiones
4  requirements.txt      <-- "Receta" con las dependencias externas y sus versiones exactas
5  pyvenv.cfg            <-- Archivo de configuración clave que define el entorno virtual
6  README.md             <-- Documentación con el propósito y uso del proyecto
7
8  .venv/                <-- Entorno virtual aislado (ligero y desechable)
9      bin/ (o Scripts/)  <-- Ejecutables de Python y pip internos del entorno
10     lib/site-packages/ <-- Donde se instalan las librerías externas para este proyecto
11
12  src/ o nombre_paquete/ <-- Carpeta principal del código fuente
13     __init__.py         <-- Identifica al directorio como un paquete jerárquico
14     modulo_principal.py  <-- Módulo con la lógica principal o punto de entrada
15     submódulo_util.py   <-- Otros módulos con funciones específicas (ej. utilidades)
16
```

```
17 tests/                                <-- Carpeta dedicada a las pruebas unitarias
18     __init__.py                       <-- Permite importar módulos de src para ser probados
19     test_logica.py                    <-- Archivos de prueba (suelen empezar con "test_")
20
21 data/ o templates/                   <-- Archivos auxiliares, como datos .csv o plantillas HT
```

- Herramientas como [Poetry](#) usan y generan esta estructura por defecto